SPECYFIKACJA TECHNICZNA V1.0

DRAFT

ClientOps

Platforma CRM nowej generacji

Architektura & Stack Technologiczny

Kompletna specyfikacja systemu CRM z panelem klienta, modułem Discovery & Brief AI, systemem SLA, edytorem dokumentów, modułem SEO AI i powiadomieniami wielokanałowymi.

DATA	WERSJA	STACK	TYP
Marzec 2026	1.6.0	Laravel + React	SaaS / Multi-tenant

Spis treści

Dokument zawiera 11 sekcji

1 Przegląd systemu i moduły

2 Stack technologiczny

3 Architektura subdomen i multi-tenancy

4 Moduł SEO z AI

5 System SLA i ticketów

6 Dokumenty — edycja online i PDF

7 Powiadomienia wielokanałowe

8 Role i uprawnienia

9 Audit log — rejestrowanie aktywności

10 Priorytety i plan wdrożenia

11 Discovery & Brief AI — moduł onboardingu projektowego

1

OVERVIEW

Przegląd systemu i moduły

ClientOps to wielomodułowy system CRM klasy SaaS przeznaczony do zarządzania klientami, projektami i działaniami marketingowymi (SEO). System oferuje dwa interfejsy: panel administracyjny dla zespołu oraz dedykowane panele klienckie dostępne pod indywidualnymi subdomenami.

CRM — Leady i Klienci

Zarządzanie lejkiem sprzedaży, kontaktami, historią komunikacji. Kanban dla leadów z drag & drop. Segmentacja klientów i tagowanie.

Projekty

Tworzenie i zarządzanie projektami powiązanymi z klientami. Zadania, kamienie milowe, śledzenie postępu. Powiązanie z SLA.

SEO z AI

Audyty techniczne, analiza słów kluczowych, semantyka treści, integracja z Google Search Console i Analytics 4. Silnik AI.

SLA i Tickety

System zgłoszeń dla klientów z SLA (czas odpowiedzi / rozwiązania). Automatyczne przekazywanie ticketów do Kanban admina.

Dokumenty

Edytor online (TipTap), wersjonowanie, eksport do PDF. Dokumenty powiązane z projektami, klientami lub ticketami.

Powiadomienia

Trzy kanały: Email, SMS (SMSAPI.pl), in-app real-time (Reverb). Preferencje per użytkownik z quiet hours dla SMS.

Panel Klienta

Dedykowana subdomena `{slug}.clientops.pl` per klient. Podgląd projektów, zgłaszanie ticketów, dostęp do dokumentów i raportów SEO.

Kanban

Ujednolicony Kanban obsługujący leady, tickety i zadania projektowe. Jeden model z polymorphic relations, drag & drop (dnd-kit).

🎙 Discovery & Brief AI

Nagranie rozmowy z klientem → transkrypcja Whisper → brief AI. Kwestionariusz z pytaniami stałymi i generowanymi przez AI w kontekście branży klienta. Analiza stron referencyjnych (Vision AI) i generowanie makiety wireframe. Integracja z modułem SEO (tryb redesign).

🔗 Flow sprzedaży

Lead → Oferta → **Discovery Session** → Projekt → zasoby projektu. Discovery jest pomostem między sprzedażą a realizacją — brief zatwierdza klient w panelu klienckim.

2

TECHNOLOGY

Stack technologiczny

Backend

TECHNOLOGIA	WERSJA	OPIS & UZASADNIENIE
Laravel	12.x	Główny framework PHP do budowy aplikacji webowych. Dostarcza routing, ORM (Eloquent), system kolejek, broadcasting, autoryzację i wiele więcej. Laravel 12 wprowadza ulepszone starter kity, streamlined middleware API oraz lepszą integrację z nowoczesnymi narzędziami frontendowymi. **Dlaczego:** Największy ekosystem PHP, najlepsza dokumentacja, aktywna społeczność. Dostarcza out-of-the-box wszystko czego wymaga CRM tej skali — bez składania frameworka z osobnych bibliotek.
PHP	8.3+	Nowoczesna wersja PHP z Fiber (asynchroniczność), named arguments, readonly properties i natywne typy wyliczeniowe (enums). Wersja 8.3 wprowadza typed class constants, readonly w klasach anonimowych oraz lepszą obsługę błędów. Wydajność PHP 8.x jest o ~30% wyższa niż PHP 7.4 dzięki JIT compiler. **Dlaczego:** Wymagany przez Laravel 12. PHP 8.3 to najnowsza stabilna wersja z długim wsparciem (LTS do 2027). Enums i typed properties kluczowe dla modelowania statusów (LeadStatus, TicketPriority).
PostgreSQL	16+	Zaawansowana relacyjna baza danych open-source z pełnym wsparciem dla ACID, JSON/JSONB, full-text search i rozszerzeń (TimescaleDB). PostgreSQL 16 wprowadza równoległe wykonywanie zapytań, lepszy query planner i wsparcie dla logicznej replikacji. Obsługuje zarówno strukturalne dane CRM jak i półstrukturalne dane audytów SEO (JSONB). **Dlaczego:** MySQL nie obsługuje TimescaleDB ani zaawansowanego JSONB — oba kluczowe dla modułu SEO. PostgreSQL jest standardem dla nowych projektów SaaS wymagających złożonych zapytań analitycznych.
Redis	7.x	Szybki in-memory data store działający jako cache, broker kolejek i magazyn sesji. Używany przez Laravel Horizon do dystrybucji jobów, przez Laravel do cache'owania zapytań i przez Reverb do przechowywania stanu połączeń WebSocket. Redis 7.x dodaje Redis Functions i ACL v2 dla lepszej kontroli dostępu. **Dlaczego:** Jeden komponent pełni trzy role (cache + queue + session) — upraszcza infrastrukturę. Alternatywy (database queue driver) nie skalują się przy obciążeniu jakie generują audyty SEO i powiadomienia.

TECHNOLOGIA	WERSJA	OPIS & UZASADNIENIE
Laravel Queues	—	Wbudowany system kolejek Laravel umożliwiający asynchroniczne wykonywanie zadań w tle — bez blokowania odpowiedzi HTTP. Joby mogą być opóźniane, ponawiane przy błędach, priorytetyzowane i rozdzielane na dedykowane kolejki (np. osobna kolejka dla AI, osobna dla emaili). Fundament dla wszystkich operacji długotrwałych w systemie. **Dlaczego:** Audyt SEO trwa minuty, generowanie PDF sekundy, wysyłka SMS może mieć opóźnienia API — bez kolejek użytkownik czekałby na odpowiedź HTTP. Queues sprawiają że każda akcja jest natychmiastowa z perspektywy użytkownika.
Laravel Sanctum	—	Lekki system autentykacji dla SPA i API oparty o cookies (stateful) lub tokeny (stateless). Dla panelu admina i panelu klienta używa autentykacji cookie-based — bezpieczniejszej niż tokeny w localStorage, odpornej na XSS. Dla potencjalnych integracji zewnętrznych obsługuje Personal Access Tokens. **Dlaczego:** Passport to overhead dla SPA (OAuth2 server). Sanctum jest dedykowany dla tego wzorca użycia: SPA na tej samej domenie + opcjonalne API tokeny. Natywna integracja z subdomenami przez SESSION_DOMAIN.
Laravel Policies / Gates	—	Wbudowany w Laravel system autoryzacji na poziomie zasobów (Policies) i akcji (Gates). Policy to klasa PHP powiązana z modelem Eloquent — definiuje kto może wykonać daną akcję na konkretnym rekordzie (np. czy ten user może edytować ten projekt). Gates to proste domknięcia dla uprawnień niezwiązanych z modelem. **Dlaczego:** Spatie Permission zarządza rolami — Policies zarządzają dostępem do konkretnych rekordów. To dwa poziomy autoryzacji: "czy masz rolę manager" (Spatie) oraz "czy jesteś przypisany do tego projektu" (Policy). Oba są konieczne.
Spatie Permission	—	Pakiet implementujący RBAC (Role-Based Access Control) dla Laravel z obsługą ról i uprawnień per guard. Pozwala przypisywać role użytkownikom, definiować granularne uprawnienia (np. tickets.respond, seo.view) i sprawdzać je przez Blade directives i Gate facades. Integruje się z Laravel's Policy system dla ochrony zasobów. **Dlaczego:** Pisanie własnego RBAC to tygodnie pracy i źródło błędów bezpieczeństwa. Spatie to standard de facto w ekosystemie Laravel — przetestowany, dobrze udokumentowany, z obsługą wielu guardów (admin vs client panel).

TECHNOLOGIA	WERSJA	OPIS & UZASADNIENIE
Laravel Notifications	—	Wbudowany w Laravel system powiadomień obsługujący wiele kanałów przez jeden interfejs: email, SMS, database (in-app), broadcast (WebSocket), Slack i inne. Każde powiadomienie to klasa PHP z metodami per kanał — jeden kod obsługuje wszystkie kanały. Preferencje użytkownika decydują które kanały są aktywne. **Dlaczego:** Alternatywa — osobny kod dla email, osobny dla SMS, osobny dla WebSocket — prowadzi do duplikacji i niespójności. Notifications unifikuje wszystko w jednym miejscu. Kluczowe dla systemu z trzema kanałami (email + SMS + in-app).
Laravel Horizon	—	Dashboard i supervisor dla kolejek Redis w Laravel, umożliwiający monitoring jobów w czasie rzeczywistym. Automatycznie balansuje workery między kolejkami na podstawie obciążenia, powiadamia o nieudanych jobach i przechowuje statystyki przepustowości. Krytyczny dla heavy asynchronicznych operacji: audyty SEO, generowanie PDF, wysyłka powiadomień. **Dlaczego:** Bez Horizon kolejki są "czarną skrzynką" — nie wiesz ile jobów czeka, które się nie powiodły, ile trwają. Horizon to observability dla kolejek, niezbędna gdy audyt SEO składa się z 6 różnych jobów uruchamianych seryjnie.
Laravel Reverb	—	Natywny serwer WebSocket dla Laravel, wprowadzony jako alternatywa dla Pusher bez zewnętrznych zależności. Obsługuje prywatne i publiczne kanały, presence channels (kto jest online) oraz skalowanie przez Redis pub/sub. Używany do in-app powiadomień real-time, aktualizacji statusu ticketów i live Kanban updates. **Dlaczego:** Pusher kosztuje przy skali i uzależnia od zewnętrznego dostawcy. Reverb działa self-hosted, jest oficjalnie wspierany przez Laravel team i nie wymaga żadnych zmian w kodzie aplikacji względem Pushera (ten sam protokół).
S3 / MinIO	—	Object storage do przechowywania plików: dokumenty, załączniki, eksportowane PDF, dane SEO. S3 (AWS) lub kompatybilny self-hosted MinIO — identyczne API, wymienne bez zmiany kodu. Laravel Filesystem abstrahuje storage przez ten sam interfejs niezależnie od wybranego drivera. Spatie Media Library korzysta z tego storage dla wszystkich mediów. **Dlaczego:** Przechowywanie plików na dysku serwera nie skaluje się i komplikuje deployment. S3/MinIO to standard — pliki są poza aplikacją, dostępne z CDN, łatwe do backupu. MinIO na start (self-hosted, zero kosztów), migracja do S3 bez zmiany kodu gdy zajdzie potrzeba.

TECHNOLOGIA	WERSJA	OPIS & UZASADNIENIE
Spatie Activity Log	—	Pakiet do rejestrowania każdej akcji w systemie — zarówno adminów jak i klientów. Zapisuje kto (causer), co (subject), kiedy i jaka była zmiana (diff old/new w JSON). Tabela `activity_log` jest wspólna dla całego systemu, logowane zdarzenia: auth (login/logout), CRM (create/update/delete/convert), Kanban (zmiana statusu), dokumenty, tickety, powiadomienia, role. **Dlaczego:** Bez audit logu nie wiadomo kto usunął kontakt, kto zmienił SLA policy, kiedy klient po raz pierwszy zalogował się do panelu. Spatie Activity Log jest de facto standardem w Laravel — działa przez trait na modelu, zero custom kodu dla podstawowych operacji CRUD.
Spatie Media Library	—	Pakiet do zarządzania plikami i mediami powiązanymi z modelami Eloquent, z obsługą konwersji i optymalizacji obrazów. Abstrahuje storage — pliki mogą być przechowywane lokalnie, na S3 lub innym dysku bez zmiany kodu aplikacji. Używany dla załączników dokumentów, plików SEO i eksportowanych PDF. **Dlaczego:** Bez Media Library każdy upload to ręczne zarządzanie ścieżkami, konwersjami, czyszczeniem. Pakiet dostarcza polymorphic relacje (każdy model może mieć media), automatyczne konwersje obrazów i responsywne URL z jedną linią kodu.
Laravel Browsershot	—	Wrapper Laravel dla biblioteki Puppeteer (Node.js), umożliwiający generowanie PDF i screenshotów przez headless Chromium. Zapewnia najwyższą jakość renderowania CSS — dokładnie to co widzi przeglądarka, przeniesione do PDF. Generowanie odbywa się asynchronicznie przez kolejkę, użytkownik dostaje powiadomienie gdy PDF jest gotowy. **Dlaczego:** DomPDF nie obsługuje flexbox/grid — dokumenty TipTap używają nowoczesnego CSS. Browsershot renderuje identycznie jak Chrome, co gwarantuje że eksportowany PDF wygląda tak samo jak podgląd w edytorze.
Laravel Scout + Meilisearch	—	Scout to abstrakcja full-text search dla Eloquent modeli, Meilisearch to szybki open-source silnik wyszukiwania z relevance ranking i typo-tolerance. Razem umożliwiają błyskawiczne wyszukiwanie w dokumentach, danych SEO i historii klientów bez obciążania PostgreSQL. Meilisearch można hostować self-hosted lub używać Meilisearch Cloud. **Dlaczego:** PostgreSQL LIKE query nie skaluje się na tysiącach dokumentów i nie obsługuje literówek ani relevance rankingu. Meilisearch zwraca wyniki w <50ms z typo-tolerance — kluczowe dla wyszukiwarki w panelu klienta i admina.

TECHNOLOGIA	WERSJA	OPIS & UZASADNIENIE
TimescaleDB	—	Rozszerzenie PostgreSQL optymalizujące przechowywanie i zapytania dla danych time-series (szeregi czasowe). Automatycznie partycjonuje tabele po czasie (hypertables), co przyspiesza zapytania z filtrem po dacie nawet 100x. Idealne dla rankingów słów kluczowych SEO, gdzie zbieramy dziesiątki tysięcy punktów danych dziennie per klient. **Dlaczego:** Tabela rankingów bez TimescaleDB przy 100 klientach × 500 słów kluczowych × 365 dni = 18 mln wierszy rocznie — standardowe zapytania stają się wolne. TimescaleDB rozwiązuje to bez zmiany stacku, bo to rozszerzenie PostgreSQL.

Frontend

TECHNOLOGIA	OPIS & UZASADNIENIE
React 19 + TypeScript	Biblioteka JavaScript do budowy interfejsów użytkownika oparta o komponenty i jednokierunkowy przepływ danych. React 19 wprowadza Server Components, Actions i ulepszony concurrent rendering dla płynniejszego UX. TypeScript dodaje statyczne typowanie, eliminując całą klasę błędów runtime i znacząco przyspieszając development w dużym projekcie. **Dlaczego:** Największy ekosystem komponentów i narzędzi. TypeScript obowiązkowy przy CRM tej skali — typy na API responses, formularze i modele eliminują całą klasę błędów jeszcze przed uruchomieniem.
Vite	Nowoczesny build tool oparty o natywne ES modules i esbuild, zapewniający błyskawiczny cold start serwera deweloperskiego (poniżej 300ms) i Hot Module Replacement bez przeładowania strony. Vite optymalizuje bundle produkcyjny przez Rollup z tree-shaking i code splitting per route. Zastępuje Webpack oferując wielokrotnie szybszy development experience. **Dlaczego:** Webpack przy projekcie tej wielkości startuje 30–60 sekund. Vite startuje poniżej sekundy. Przy kilkuset komponentach różnica w DX jest znacząca każdego dnia pracy.
Tailwind CSS v4	Utility-first CSS framework gdzie style definiuje się bezpośrednio w klasach HTML, eliminując przełączanie się między plikami CSS i JSX. Tailwind v4 to kompletne przepisanie silnika — oparty o Lightning CSS (Rust), do 10x szybszy build. Konfiguracja wyłącznie przez `@tailwindcss/vite` plugin, bez pliku konfiguracyjnego JS. **Dlaczego:** Fundament design systemu — phoenix-design-system definiuje tokeny przez `@theme` Tailwinda v4. Utility-first eliminuje problem nazewnictwa klas CSS i dead code w dużym projekcie.

TECHNOLOGIA	OPIS & UZASADNIENIE
phoenix-design-system	Własny design system przepisany z ClientOps Bootstrap na Tailwind v4. Zawiera tokeny (`@theme` : kolory, typografia, spacing, shadows, border-radius, dark mode przez CSS variables) oraz gotowe komponenty CSS (buttons, badges, cards, forms, modal, dropdown, kanban, timeline, wizard i inne). Jeden import (`@import "./phoenix-design-system/index.css"`) daje pełny system. **Dlaczego:** Zastępuje shadcn/ui — mamy już wszystkie komponenty ze spójnym wyglądem dopasowanym do projektu. shadcn/ui byłoby nadmiarowe i wymagałoby nadpisywania styli. Migracja z klas CSS do React+cva odbywa się stopniowo.
class-variance-authority (cva)	Biblioteka do tworzenia typowanych wariantów komponentów React — definiujesz raz warianty (variant, size, intent) i dostajesz w pełni typowaną funkcję generującą klasy Tailwind. Integruje się z `clsx` i `tailwind-merge` dla obsługi warunkowych i nadpisanych klas. Pozwala stopniowo migrować komponenty CSS z phoenix-design-system do natywnych komponentów React. **Dlaczego:** Bez cva każdy komponent Button/Badge wymaga ręcznego zarządzania klasami warunkowymi — chaos przy wielu wariantach. cva daje type safety: TypeScript wie jakie warianty istnieją i pilnuje poprawności props.
TanStack Query v5	Biblioteka do zarządzania stanem serwera w React — fetching, caching, synchronizacja i aktualizacja danych z API. Automatycznie obsługuje loading states, error handling, background refetching i optimistic updates, eliminując setki linii boilerplate kodu. TanStack Query v5 wprowadza ulepszony infinite scrolling, streamlined API i lepszy TypeScript inference. **Dlaczego:** W CRM niemal każdy widok to dane z serwera. Bez TanStack Query każdy komponent potrzebuje własnego useEffect + useState + loading + error. TanStack zastępuje to kilkoma liniami z automatycznym cache invalidation.
React Hook Form + Zod	React Hook Form to biblioteka formularzy minimalizująca re-rendery (uncontrolled components), Zod to TypeScript-first schema validation z pełnym type inference. Razem tworzą potężny duet — Zod definiuje schemat danych, Hook Form obsługuje formularz, walidacja dzieje się automatycznie z precyzyjnymi komunikatami błędów. Jeden schemat Zod może być reużyty zarówno na frontendzie jak i w API. **Dlaczego:** CRM ma dziesiątki formularzy (lead, klient, projekt, ticket, oferta...). Formik jest wolniejszy, własna walidacja to duplikacja logiki backendowej. Jeden Zod schema obsługuje frontend i może być współdzielony z backendem.
dnd-kit	Nowoczesna, lekka biblioteka drag & drop dla React z pełnym wsparciem dla TypeScript i dostępności (keyboard DnD). Modułowa architektura pozwala budować pionowe listy, siatki i kanban boards bez zbędnego kodu. Obsługuje touch events, sensory (mouse/touch/keyboard) i smooth animations przez CSS transforms bez layout thrashing. **Dlaczego:** React Beautiful DnD nie jest już aktywnie rozwijany i ma problemy z React 18+. dnd-kit to jego następca — lżejszy, szybszy. Kluczowy dla Kanban leadów i ticketów.

TECHNOLOGIA	OPIS & UZASADNIENIE
TipTap v2	Headless rich text editor oparty o ProseMirror, w pełni kontrolowalny przez React i TypeScript, bez narzuconych styli — aplikujesz klasy phoenix-design-system bezpośrednio. Obsługuje tabele, listy, obrazy, mentions (@user), linki i można go rozszerzyć o custom nodes (np. bloki AI suggestions). Zapisuje treść jako JSON lub HTML, co ułatwia eksport do PDF przez Browsershot. **Dlaczego:** TinyMCE wymaga API key, ma narzucone style trudne do nadpisania i jest ciężki. TipTap jest headless (MIT) — stylujemy go phoenix-design-system bez walki z biblioteką. JSON output to lepszy format niż HTML dla wersjonowania dokumentów.
echarts + echarts-for-react	Apache ECharts to jedna z najpotężniejszych bibliotek wykresów — obsługuje linie, słupki, pie, scatter, heatmapy, gauge, tree charts, mapy i wiele więcej. `echarts-for-react` to oficjalny wrapper React. Wykresy są w pełni konfigurowalane przez opcje JSON i responsywne out of the box. **Dlaczego:** Moduł SEO wymaga heatmap (godziny aktywności), gauge (score audytu), tree chart (struktura strony) — Recharts tego nie ma. ECharts zastępuje Recharts i Tremor jednocześnie, z większymi możliwościami i spójnym API.
React Router v7	Standard routingu dla React SPA z pełnym wsparciem dla nested routes, lazy loading (code splitting per route) i data loading patterns. React Router v7 unifikuje się z Remix oferując loaders i actions. Code splitting zapewnia że klient pobiera tylko kod potrzebny dla aktualnego widoku. **Dlaczego:** De facto standard — każdy React developer go zna, co ułatwia onboarding. Code splitting krytyczny przy dużej aplikacji z dziesiątkami widoków i modułów.
Zustand	Minimalistyczna biblioteka do zarządzania globalnym stanem aplikacji (~1KB gzip), znacznie prostsza niż Redux bez boilerplate. Używana dla stanu który nie należy do serwera (TanStack Query) — aktywne powiadomienia, preferencje UI, stan sidebara, dane zalogowanego użytkownika. Nie wymaga Provider wrappera i działa poza komponentami React. **Dlaczego:** Redux to overhead dla stanu UI. React Context re-renderuje cały subtree. Zustand subskrybuje tylko do konkretnych wartości — idealne dla powiadomień real-time z Reverb.
Laravel Echo	JavaScript biblioteka ułatwiająca subskrypcję kanałów WebSocket w Laravel, współpracująca natywnie z Reverb. Abstrahuje niskopoziomowe WebSocket API — prosty interfejs do nasłuchiwania eventów na prywatnych i publicznych kanałach. Używana do real-time powiadomień in-app, live aktualizacji ticketów i Kanban. **Dlaczego:** Pisanie własnego WebSocket klienta dla Laravel Broadcasting to niepotrzebna praca. Echo obsługuje automatycznie autoryzację prywatnych kanałów i reconnection logic.

TECHNOLOGIA	OPIS & UZASADNIENIE
react-select	Zaawansowany komponent select z wbudowanym multi-select, autocomplete, async loading opcji i pełną customizacją. Zastępuje natywny HTML select i Choices.js z ClientOps Bootstrap. Styl nadpisywalny przez `classNames` API — kompatybilny z phoenix-design-system. **Dlaczego:** Natywny select HTML nie obsługuje multi-select z wyszukiwaniem. react-select jest standardem dla tego wzorca w React — używany m.in. w formularzu leadów (tagi) i przypisywaniu użytkowników do projektów.
react-dropzone	Hook i komponent do obsługi drag & drop uploadu plików z walidacją typów, rozmiarów i podglądem. Integruje się z Spatie Media Library po stronie backendu. Używany w module dokumentów (załączniki) i profilu klienta. **Dlaczego:** Natywny input file nie obsługuje drag & drop. react-dropzone to lekki, elastyczny hook — łatwy do ostylowania komponentami phoenix-design-system.
react-day-picker	Lekki, elastyczny date picker i date range picker dla React, bez zewnętrznych zależności. Styl w pełni kontrolowany przez CSS classes — kompatybilny z Tailwind. Używany w formularzach leadów (follow-up date), projektów (deadline) i filtrach tabeli. **Dlaczego:** flatpickr z ClientOps Bootstrap nie integruje się z React Hook Form. react-day-picker jest headless jak TipTap — aplikujemy klasy phoenix-design-system bezpośrednio.
@fullcalendar/react	Pełnofunkcjonalny kalendarz eventów z widokami miesiąc/tydzień/dzień/lista i drag & drop eventów. Oficjalny wrapper React z TypeScript support. Używany w module projektów (harmonogram zadań) i CRM (follow-upy, spotkania). **Dlaczego:** Bezpośredni odpowiednik fullcalendar z ClientOps Bootstrap — ta sama biblioteka, oficjalny wrapper React, zero nauki od zera. Potrzebny dla widoku kalendarza CRM i harmonogramu projektów.
lucide-react	Biblioteka ikon SVG jako komponenty React — następca Feather Icons, przepisany w TypeScript. Ponad 1000 ikon w spójnym stylu outline, każda jako osobny named export (tree-shakeable). Używana w całym interfejsie. **Dlaczego:** feather-icons i FontAwesome wymagają osobnych wrapperów. lucide-react jest natywnie dla React, tree-shakeable (zero nieużywanych ikon w bundle) i ma aktywny rozwój.
dayjs + lodash-es	dayjs to lekka (~2KB) biblioteka do manipulacji datami (format, diff, parse, relative time) — bezpośredni zamiennik moment.js. lodash-es to tree-shakeable wersja lodash z utility funkcjami (debounce, throttle, groupBy, pick). **Dlaczego:** Obie biblioteki były używane w ClientOps Bootstrap i działają w React bez żadnych zmian. lodash-es zamiast lodash — tree-shaking eliminuje nieużywane funkcje z bundle.

Dlaczego React SPA zamiast Inertia.js?

Panel klienta na subdomenach (`{slug}.clientops.pl`) wymaga oddzielnego frontendu, co oznacza konieczność budowania API. Skoro API jest i tak potrzebne, React SPA jest naturalnym wyborem — oba

> frontendy (admin + client) mogą współdzielić kod i korzystać z tego samego API.

3

ARCHITECTURE

Architektura subdomen i multi-tenancy

Struktura subdomen

SUBDOMENA	TYP	OPIS
`app.clientops.pl`	Admin Panel	CRM dla zespołu — leady, projekty, SEO, kanban, ustawienia
`api.clientops.pl`	API	Laravel REST API — obsługuje oba frontendy + potencjalne integracje
`{slug}.clientops.pl`	Client Portal	Dedykowany panel per klient — projekty, tickety, dokumenty, SEO

Identyfikacja tenanta

Middleware wyciąga slug z subdomeny i wstrzykuje klienta do IoC container:

```php
// app/Http/Middleware/IdentifyTenant.php
public function handle(Request $request, Closure $next)
{
    $host = $request->getHost();         // acme.clientops.pl
    $slug = explode('.', $host)[0];     // acme

    if ($slug === 'app' || $slug === 'api') {
        return $next($request);            // admin / api — skip
    }

    $tenant = Client::where('slug', $slug)->firstOrFail();
    app()->instance(Client::class, $tenant);
    app()->instance('tenant', $tenant);

    return $next($request);
}
```

Konfiguracja Sanctum dla wildcard subdomen

```
# .env
SESSION_DOMAIN=.clientops.pl
SANCTUM_STATEFUL_DOMAINS=app.clientops.pl,*.clientops.pl

# config/cors.php
'allowed_origins_patterns' => ['#^https://[a-z0-9-]+\.clientops\.pl$#'],
'supports_credentials' => true,
```

Multi-tenancy — model danych

Wybór: Single Database z Global Scopes

Dla tego projektu wystarczy jedna baza danych z `client_id` na każdej tabeli oraz automatycznym Global Scope. Prostsze w utrzymaniu i wystarczające na start. DB per tenant to nadmiarowa złożoność bez wymiernych korzyści na tym etapie.

```
// Trait dla wszystkich modeli powiązanych z klientem
trait BelongsToTenant
{
    protected static function booted(): void
    {
        static::addGlobalScope(new TenantScope);

        static::creating(function ($model) {
            $model->client_id ??= app('tenant')?->id;
        });
    }
}
```

Slug — identyfikator zamiast ID w URL

W całym systemie zasoby są identyfikowane przez **slug** zamiast numerycznego ID. Dotyczy to klientów, projektów, leadów, ofert, dokumentów i audytów.

ENCJA	PRZYKŁAD URL	UNIQUE SCOPE
clients	`/clients/dcg-wroclaw`	globalnie (subdomena)
projects	`/projects/strona-www-2026`	per `client_id`
leads	`/leads/dcg-wroclaw`	per `client_id` (tenant)
offers	`/offers/oferta-seo-q1-2026`	per `client_id`
documents	`/documents/umowa-o-wspolpracy`	per `documentable` (projekt/klient)

ENCJA	PRZYKŁAD URL	UNIQUE SCOPE
audits	`/audits/audyt-seo-marzec-2026`	per `project_id`

Polityka slug — 4 zasady

1. Slug generowany automatycznie z tytułu/nazwy przy tworzeniu rekordu — `spatie/laravel-sluggable`
2. Slug jest **immutable** — zmiana tytułu nie aktualizuje slugu
3. Kolizja w tym samym scope → auto-suffix: `umowa` → `umowa-2` → `umowa-3`
4. Ręczna zmiana slugu (admin) → stary slug zapisywany w `slug_redirects` → `301 Redirect`

Autentykacja — metody logowania

System obsługuje trzy ścieżki uwierzytelniania, wszystkie oparte na Sanctum (cookies):

METODA	PAKIET	OPIS
Email + hasło	Laravel Sanctum	Standardowe logowanie. Walidacja credentiali, wystawienie sesji cookie.
Google OAuth	`laravel/socialite`	Redirect do Google → callback → `users.google_id`. Przy pierwszym logowaniu tworzy konto automatycznie. Konto Google nie wymaga hasła w systemie.
2FA przez email	własna implementacja	Opcjonalne, włączane przez użytkownika w Ustawieniach. Po poprawnym haśle — kod 6-cyfrowy wysyłany mailem, ważny 10 min. Wymaga potwierdzenia przed ustawieniem sesji.

Google OAuth — przepływ

```
// Przekierowanie do Google
GET /auth/google/redirect
→ return Socialite::driver('google')→redirect();

// Callback po autoryzacji
GET /auth/google/callback
$googleUser = Socialite::driver('google')→user();

$user = User::firstOrCreate(
    ['google_id' ⇒ $googleUser→id],
    [
        'name'  ⇒ $googleUser→name,
        'email' ⇒ $googleUser→email,
        'password' ⇒ null,     // brak hasła dla kont OAuth
    ]
);

Auth::login($user);          // wystawia sesję Sanctum
return redirect('https://app.clientops.pl/dashboard');
```

Kolumny dodane do tabeli `users`: `google_id (nullable)`, `password (nullable)`. Użytkownik bez hasła nie może logować się przez formularz — tylko przez Google.

2FA przez email — przepływ

```
// Kolumny w tabeli users:
two_factor_enabled    boolean   default false
two_factor_code       varchar   nullable
two_factor_expires_at timestamp nullable

// Krok 1: walidacja hasła → jeśli 2FA włączone
$code = random_int(100000, 999999);
$user→update([
    'two_factor_code'       ⇒ Hash::make($code),
    'two_factor_expires_at' ⇒ now()→addMinutes(10),
]);
Notification::send($user, new TwoFactorCodeNotification($code));
return response()→json(['requires_2fa' ⇒ true]);

// Krok 2: weryfikacja kodu
if (Hash::check($request→code, $user→two_factor_code)
    && $user→two_factor_expires_at→isFuture()) {
    $user→update(['two_factor_code' ⇒ null, 'two_factor_expires_at' ⇒ null]);
    Auth::login($user);
    return response()→json(['authenticated' ⇒ true]);
}
```

> **2FA jest opcjonalne**

— użytkownik włącza je sam w zakładce Ustawienia → Bezpieczeństwo. Po włączeniu przy każdym logowaniu (hasłem) wymagany jest kod email. Logowanie przez Google omija 2FA (Google sam zapewnia MFA). Kod wygasa po 10 minutach; próba użycia wygasłego kodu zwraca błąd i wymaga ponownego logowania.

4

MODULE

Moduł SEO z AI

Najcięższy moduł systemu. Dane napływają z wielu źródeł asynchronicznie, przetwarzane są przez kolejki (Horizon), a wyniki przechowywane w PostgreSQL z rozszerzeniem TimescaleDB dla danych time-series (rankingi).

Źródła danych

Google Search Console

Pozycje, kliknięcia, impressions, CTR per zapytanie i strona. OAuth 2.0. → `seo_keyword_ranks`, `seo_monthly_snapshots`

Google Analytics 4

Ruch organiczny, konwersje, bounce rate, engagement. GA4 Data API. → `seo_monthly_snapshots`

Własny Crawler

Błędy techniczne: broken links, meta tags, H1, duplicate content. → `seo_audit_pages`, `seo_issues`, `audit_issues`

PageSpeed Insights API

LCP, CLS, INP, FCP, TTFB per URL — dane polowe i lab. → `seo_cwv_snapshots`, `audit_scores`

Backlinks API

Referring domains, DR/DA, anchor texty, toxic links. Ahrefs API lub Semrush. → `seo_backlinks`

Crawler konkurencji

Rankingi konkurentów na wspólnych frazach, content gap analysis. → `seo_competitor_ranks`, `seo_keyword_opportunities`

AI (Claude / GPT)

Analiza semantyczna treści, content gap, sugestie optymalizacji. → `seo_recommendations`, `seo_keyword_opportunities`

Security & UX Checks

HTTPS, security headers, WCAG kontrast, alt tagi, mobile UX. → `audit_issues`, `audit_scores`

Architektura kolejkowa

```
AuditRequestedEvent
    ↓
CrawlWebsiteJob           queue: crawlers → seo_audit_pages, seo_issues, audit_issues
    ↓
FetchGSCDataJob           queue: google-apis → seo_keyword_ranks (hypertable)
FetchGA4DataJob           queue: google-apis (równolegle) → seo_monthly_snapshots
FetchPageSpeedJob         queue: google-apis (równolegle) → seo_cwv_snapshots (hypertable)
FetchBacklinksJob         queue: external-apis → seo_backlinks, seo_monthly_snapshots
FetchCompetitorRanksJob   queue: crawlers → seo_competitor_ranks (hypertable)
    ↓
RunSecurityChecksJob      queue: crawlers (tylko type: security|full) → audit_issues, audit_scores
RunUXChecksJob            queue: crawlers (tylko type: ux|full) → audit_issues, audit_scores
    ↓
AnalyzeKeywordsJob        queue: ai-processing → seo_keywords, seo_keyword_opportunities
AnalyzeSemanticsJob       queue: ai-processing (równolegle) → seo_recommendations
    ↓
GenerateAuditReportJob    audit_scores → audits.overall_score = avg(scores), PDF
    ↓
NotifyAuditCompleteJob    email + in-app notification (type: audit_complete)
```

Schemat bazy danych

```
-- Rdzeń audytu SEO (istniejące)
seo_audits                -- meta audytu: status, overall_score, created_at
seo_audit_pages           -- wyniki per URL (title, H1, meta, linki, treść)
seo_issues                -- błędy SEO: type, severity, url, assigned_to, due_date
seo_keywords              -- słowa kluczowe per projekt z target_url
seo_keyword_ranks         -- TIME-SERIES ①: pozycje, clicks, ctr (TimescaleDB)
seo_recommendations       -- sugestie AI: priorytet, status wdrożenia

-- Nowe tabele — audyt kliencki i ogólny (v2.0)
seo_monthly_snapshots     -- KPI m/m: sessions, conversions, top10_count, backlinks delta
seo_cwv_snapshots         -- TIME-SERIES ②: LCP/CLS/INP/FCP/TTFB per URL (TimescaleDB)
seo_backlinks             -- referring_domain, DR, anchor, first/lost_at
seo_competitors           -- domeny konkurentów monitorowane per projekt
seo_competitor_ranks      -- TIME-SERIES ③: pozycje konkurentów per fraza (TimescaleDB)
seo_keyword_opportunities -- content gap: frazy bez strony, z pozycjami konkurentów
audit_issues              -- generyczny issue dla ux|security|performance|content
audit_scores              -- wyniki 0-100 per kategoria → audits.overall_score = avg()
```

3 hypertable TimescaleDB

`seo_keyword_ranks`, `seo_cwv_snapshots` i `seo_competitor_ranks` to hypertable TimescaleDB partycjonowane po czasie. Umożliwiają trend pozycji, CWV history per URL i porównanie rankingów konkurentów m/m — wszystko jako zapytania time-series. TimescaleDB = rozszerzenie PostgreSQL, nie zmienia stacku.

5

MODULE

System SLA i ticketów

Dwa powiązane podsystemy: (1) tickety zgłaszane przez klientów z automatycznym SLA tracking oraz (2) deadline monitoring dla projektów. Oba zasilają Kanban admina.

Flow ticketu

Klient zgłasza ticket → status: new → Auto → Kanban task → SLA timer start → Admin odpowiada → Klient powiadomiony

Statusy ticketu

new | open | in_progress | waiting_for_client | resolved | closed

Schemat bazy danych

```
tickets
  - id, client_id, project_id
  - priority: low | medium | high | critical
  - status: new | open | in_progress | waiting | resolved | closed
  - sla_response_due_at       -- deadline pierwszej odpowiedzi
  - sla_resolution_due_at     -- deadline rozwiązania
  - first_responded_at        -- kiedy SLA response zostało spełnione
  - kanban_card_id            -- powiązanie z Kanban

sla_policies
  - client_id (lub global default)
  - priority                  -- per priorytet ticketu
  - response_time_hours       -- np. critical=2h, high=8h, low=48h
  - resolution_time_hours
  - business_hours_only: bool

ticket_comments
  - ticket_id, author_id, author_type
  - is_internal: bool         -- notatki niewidoczne dla klienta
  - content
```

Ujednolicony Kanban

Jedna tabela obsługuje różne typy kart dzięki polymorphic relations:

```
kanban_cards
  - board_id, column_id, position
  - cardable_type: "Lead" | "Ticket" | "ProjectTask"
  - cardable_id
  - assigned_to
  - due_at
```

SLA breach monitoring

```php
// Scheduled job — co 15 minut
class CheckSlaBreachesJob
{
    public function handle(): void
    {
        $breaching = Ticket::query()
            ->whereNull('first_responded_at')
            ->where('sla_response_due_at', '<', now()->addMinutes(30))
            ->get();

        $breaching->each(fn($ticket) =>
            $ticket->assignees->each->notify(new SlaBreachWarningNotification($ticket))
        );
    }
}
```

6

MODULE

Dokumenty — edycja online i PDF

Stack edytora

TipTap v2 — edytor

Najlepszy wybór dla React + TypeScript. Oparty o ProseMirror.

Obsługuje: tabele, listy, nagłówki, obrazy, mentions (@user), formatowanie, linki. Rozszerzalny — możesz dodać AI suggestions jako custom extension (np. sugestie treści SEO w dokumentach).

Laravel Browsershot — PDF

Renderuje PDF przez Chromium (Puppeteer). Najwyższa jakość CSS.

Wymaga Node.js + Chromium na serwerze. Generowanie asynchroniczne przez kolejkę — użytkownik dostaje powiadomienie gdy PDF jest gotowy.

Flow dokumentu

Tworzenie → Edycja TipTap → Zapis JSON + HTML → Wersja snapshot

Export PDF → GeneratePDFJob → S3 / Storage → Link + Powiadomienie

Schemat bazy danych

```
documents
  - documentable_type: "Project" | "Client" | "Ticket"   -- polymorphic
  - documentable_id
  - title, slug
  - content_json     -- TipTap JSON format (do edycji)
  - content_html     -- wygenerowany HTML (do renderowania)
  - pdf_path         -- ścieżka ostatniego PDF w storage
  - version: int
  - created_by, updated_by

document_versions
  - document_id
  - version: int
  - content_json     -- snapshot przy każdym ważnym zapisie
  - created_by, created_at
```

Generowanie PDF — kod kolejki

```php
class GenerateDocumentPdfJob implements ShouldQueue
{
    public function handle(Browsershot $browsershot): void
    {
        $html = view('pdf.document', [
            'document' => $this->document,
            'client'   => $this->document->client,
        ])->render();

        $path = "documents/{$this->document->id}/v{$this->document->version}.pdf";

        $browsershot->html($html)
            ->format('A4')
            ->margins(20, 20, 20, 20)
            ->save(storage_path("app/{$path}"));

        $this->document->update(['pdf_path' => $path]);

        $this->document->creator->notify(new PdfReadyNotification($this->document));
    }
}
```

7

MODULE

Powiadomienia wielokanałowe

Trzy kanały

Email

Laravel Mail + queued. Szablony Blade per typ zdarzenia. Markdown mail dla prostych, custom HTML dla brandowanych.

SMS

SMSAPI.pl (PL) lub Twilio (global). Custom Laravel Notification channel. Quiet hours — brak SMS w nocy. Tylko dla high-priority zdarzeń.

In-App (Real-time)

Laravel Echo + Reverb (WebSocket). Powiadomienia w czasie rzeczywistym. Persist w DB dla historii. Broadcast na prywatnym kanale usera.

Typy zdarzeń → kanały

ZDARZENIE	EMAIL	SMS	IN-APP
Nowy ticket	✅	🔶 (critical)	✅
Odpowiedź na ticket	✅	—	✅
SLA breach warning	✅	✅	✅
Audyt SEO ukończony	✅	—	✅
PDF gotowy	—	—	✅
Deadline projektu (24h)	✅	✅	✅
Nowy dokument	✅	—	✅

Preferencje powiadomień

```
notification_preferences
  - user_id
  - notification_type: ticket_update | sla_breach | audit_complete | pdf_ready | deadline | mention | messag
  - channels: ['mail', 'sms', 'database']    -- JSON array
  - quiet_hours_from: '22:00'
  - quiet_hours_to:   '08:00'
  - quiet_hours_timezone: 'Europe/Warsaw'
```

Implementacja notyfikacji

```
class TicketUpdatedNotification extends Notification
{
    public function via($notifiable): array
    {
        return $notifiable->activeChannelsFor('ticket_update');
        // zwraca ['mail', 'database', 'broadcast'] lub ['sms'] itd.
    }

    public function toMail($notifiable): MailMessage { ... }
    public function toSms($notifiable): SmsMessage { ... }
    public function toDatabase($notifiable): array { ... }
    public function toBroadcast($notifiable): BroadcastMessage { ... }
}
```

Zunifikowany inbox — System 2: wiadomości wewnętrzne

Oprócz notyfikacji systemowych (SLA, @mention, ticket) użytkownicy mogą wysyłać wiadomości do innych adminów lub do klientów bezpośrednio w CRM — bez wychodzenia poza system. Oba strumienie trafiają do **jednego inboxu**.

Notyfikacje systemowe

SLA breach, @mention, nowy ticket, task assigned, odpowiedź na komentarz, PDF gotowy. Generowane automatycznie przez system.

Wiadomości admin → admin

Admin wysyła wiadomość do innego admina/managera/pracownika. Zostaje w CRM, opcjonalnie kopia na email.

Wiadomości admin → klient

Admin wysyła wiadomość do klienta (client-admin lub client-user). Klient widzi ją w swoim panelu na subdomenie.

Jak to działa — flow

```
// Zdarzenie systemowe (np. SLA breach)
CheckSlaBreachesJob::dispatch()
  → tworzy SlaIncident
  → wywołuje SlaBreachNotification
  → zapisuje do: notifications (type: sla_breach)
  → broadcast → Reverb → bell w UI

// Wiadomość admin → klient
Admin wysyła wiadomość
  → zapis do: messages (treść, sender, conversation_id)
  → tworzy: notifications (type: message, notifiable: klient)
  → broadcast → Reverb → bell klienta w panelu
  → opcjonalnie: email (wg notification_preferences)

// @mention w komentarzu do ticketu
Admin pisze: "@Jan sprawdź proszę"
  → zapis do: ticket_comments
  → zapis do: mentions (mentionable: comment, mentioned: Jan)
  → tworzy: notifications (type: mention, notifiable: Jan)
  → broadcast → bell Jana
```

Schematy tabel

```
-- Główna tabela notyfikacji (Laravel built-in, rozszerzona)
notifications
  id, type          -- sla_breach | mention | message | task_assigned | comment_reply | ...
  notifiable_type   -- "User" | "Client"
  notifiable_id
  data              -- JSON: { body, sender_id, link, subject_type, subject_id }
  read_at, created_at

-- Wątki wiadomości
conversations
  id, subject (nullable), created_by, created_at

conversation_participants
  conversation_id
  participable_type  -- "User" | "Client"
  participable_id
  last_read_at       -- do liczenia nieprzeczytanych

messages
  id, conversation_id
  sender_type        -- "User" | "Client"
  sender_id, body, created_at

-- @oznaczenia w komentarzach i wiadomościach
mentions
  mentionable_type   -- "Message" | "TicketComment" | "DocumentComment"
  mentionable_id
  mentioned_type     -- "User" | "Client"
  mentioned_id, created_at
```

Jeden inbox w UI — zasada działania

ŹRÓDŁO	TRAFIA DO NOTIFICATIONS?	PO KLIKNIĘCIU OTWIERA	EMAIL?
SLA breach	✅ `type: sla_breach`	Widok naruszenia SLA	✅ + SMS
Nowy ticket	✅ `type: new_ticket`	Widok ticketu	✅
@mention	✅ `type: mention`	Komentarz w kontekście (ticket/projekt)	wg preferencji
Task assigned	✅ `type: task_assigned`	Zadanie w projekcie / Kanban	wg preferencji
Wiadomość od admina	✅ `type: message`	Wątek konwersacji (inbox)	wg preferencji

ŹRÓDŁO	TRAFIA DO NOTIFICATIONS?	PO KLIKNIĘCIU OTWIERA	EMAIL?
Wiadomość do klienta	✅ `type: message`	Wątek w panelu klienta	wg preferencji
PDF gotowy	✅ `type: pdf_ready`	Link do pobrania PDF	—

Frontend — biblioteki

BIBLIOTEKA	ZASTOSOWANIE
Laravel Echo	WebSocket client — nasłuchuje na prywatnym kanale usera (`private-user.{id}`), odbiera broadcast notyfikacji w czasie rzeczywistym
Zustand	Globalny store: licznik nieprzeczytanych notyfikacji, lista ostatnich alertów — dostępny z bell icon w navbar bez prop drillingu
TanStack Query	Pobieranie listy notyfikacji i wątków konwersacji z API, invalidacja cache po odebraniu broadcast eventu
@floating-ui/react	Dropdown panel notyfikacji przy bell icon (pozycjonowanie, dismiss on outside click)
TipTap v2 (opcjonalnie)	Edytor wiadomości z obsługą @mention — autocomplete użytkowników i klientów, highlight oznaczeń

Kluczowa zasada:

Cokolwiek wymaga uwagi użytkownika → trafia do tabeli `notifications` → broadcast przez Reverb → pojawia się w jednym bell icon. Różnica jest tylko w tym co otwiera kliknięcie: link do obiektu (ticket, lead) lub wątek konwersacji. Użytkownik ma jedno miejsce — nie kilka "dzwonków".

8

SECURITY

Role i uprawnienia

System uprawnień oparty o Spatie Laravel Permission (RBAC). Sześć ról z hierarchią dostępu — dwie po stronie admina, cztery po stronie klienta.

Hierarchia ról

`super-admin`	Dostęp do wszystkiego. Zarządzanie ustawieniami systemu, konfiguracją subdomen, planami klientów, integracjami API (Google, AI). Tylko właściciel systemu.
`admin`	Pełny dostęp do CRM — leady, klienci, projekty, SEO audyty, dokumenty. Zarządzanie zespołem, przeglądanie wszystkich ticketów, konfiguracja SLA.
`manager`	Zarządza przypisanymi projektami i klientami. Może tworzyć dokumenty i zadania, obsługiwać tickety. Bez dostępu do ustawień systemu i finansów.
`employee`	Widzi tylko przypisane projekty i zadania. Może komentować tickety i tworzyć dokumenty w ramach swoich projektów. Brak dostępu do danych innych klientów.
`client-admin`	**(Panel klienta)** Pełny dostęp do panelu swojego klienta. Widzi projekty, SEO raporty, dokumenty, może zarządzać użytkownikami swojej organizacji i zgłaszać tickety.
`client-user`	**(Panel klienta)** Ograniczony dostęp — tylko podgląd przypisanych projektów i dokumentów, zgłaszanie ticketów. Bez dostępu do raportów finansowych i SEO.

Izolacja danych między klientami

Global Scope jako pierwsza linia obrony

Każdy model z trait `BelongsToTenant` automatycznie filtruje dane po `client_id`. Dodatkowo Gate policies sprawdzają uprawnienia na poziomie pojedynczego zasobu. Dwupoziomowa ochrona: scope na poziomie modelu + policy na poziomie kontrolera.

```php
// Przykład Policy
class ProjectPolicy
{
    public function view(User $user, Project $project): bool
    {
        // Global scope już odfiltrował inne klientów
        // Tu sprawdzamy czy user ma dostęp do tego projektu
        return $user->hasRole(['admin', 'manager'])
            || $project->assignees->contains($user);
    }
}
```

9

AUDIT TRAIL

Audit log — rejestrowanie aktywności

Każda akcja wykonana przez admina lub klienta jest zapisywana w tabeli `activity_log`. Log rejestruje kto wykonał akcję, na jakim obiekcie, kiedy i co się zmieniło (diff). Używany do audytu bezpieczeństwa, historii zmian i debugowania.

Schemat tabeli activity_log

```
-- Tabela zarządzana przez spatie/laravel-activitylog
activity_log
  id
  log_name          -- "auth" | "crm" | "client" | "notifications" | "system"
  description       -- "created" | "updated" | "deleted" | "login" | "converted_to_client" | ...
  subject_type      -- polymorphic: "Lead" | "Client" | "Contact" | "Offer" | ...
  subject_id
  causer_type       -- "User" (admin/manager/employee) | "Client" (client-admin/client-user)
  causer_id
  properties        -- JSON: { old: {...}, attributes: {...} } — diff zmian
  created_at
```

Co jest logowane — panel admina

LOG_NAME	ZDARZENIA
auth	login, logout, nieudane logowanie, zmiana hasła, nadanie/odebranie roli, założenie konta użytkownika
crm	Tworzenie/edycja/usuwanie: Lead, Client, Contact, Offer, Project Zmiana statusu leadu (Kanban — kolumna źródłowa i docelowa) Konwersja leadu na klienta (`converted_to_client`) Przypisanie SLA Policy do klienta/projektu
documents	Utworzenie dokumentu, zapis wersji, eksport do PDF (kto, kiedy), usunięcie dokumentu
tickets	Utworzenie ticketu, zmiana statusu, dodanie komentarza (w tym notatek wewnętrznych), zamknięcie
notifications	Wysłanie powiadomienia (typ, kanał: email/SMS/in-app, odbiorca)
system	Zmiana SLA Policy, zmiana konfiguracji, zmiany ustawień integracji (GSC, GA4, SMSAPI)

Co jest logowane — panel klienta

LOG_NAME	ZDARZENIA
auth	login do panelu klienta, logout, nieudane logowanie, zmiana hasła
client	Wyświetlenie dokumentu, pobranie PDF Utworzenie ticketu, dodanie komentarza do ticketu Odczyt powiadomienia

Czego się NIE loguje

> **Pominięte w audit logu (zbyt duży wolumen lub brak wartości):**
>
> - `seo_keyword_ranks` — TimescaleDB, miliony wpisów time-series
> - `sla_incidents` — tworzone automatycznie przez system, nie przez użytkownika
> - Odczyt listy (GET /api/leads) — nie każde przeglądanie wymaga logu

Dostęp do logów w UI

Panel admina: zakładka **"Historia zmian"** przy każdym rekordzie (Lead, Client, Project, Document, Ticket) — wyświetla ostatnie N wpisów z `activity_log` dla danego `subject_type/subject_id`. Widok globalny w menu **Ustawienia → Audit log** (tylko rola `super-admin` / `admin`) z filtrowaniem po użytkowniku, typie zdarzenia i zakresie dat.

10

ROADMAP

Priorytety i plan wdrożenia

Rekomendowana kolejność budowania systemu — od fundamentów do modułów specjalistycznych. SEO jako ostatni, bo wymaga działających klientów, projektów i integracji zewnętrznych.

1

Core — Fundament systemu

Auth (Sanctum + Spatie Permission), multi-tenancy (subdomain middleware + global scope), moduł klientów i projektów, podstawowe CRUD, panel admina (React SPA szkielet), panel klienta (szkielet na subdomenach). *Bez tego nic innego nie działa.*

2

CRM — Leady i Kanban

Moduł leadów, pipeline sprzedaży, Kanban board (dnd-kit), historia kontaktów. Ujednolicony model `kanban_cards` z polymorphic relations — fundament dla ticketów.

3

SLA i Tickety

System zgłoszeń w panelu klienta, SLA policies, SLA timer, automatyczne tworzenie kart Kanban z ticketów, breach monitoring (scheduled job co 15 min), historia komentarzy z notatkami wewnętrznymi.

4

Dokumenty

Edytor TipTap, zapis JSON/HTML, wersjonowanie (document_versions), Spatie Media Library dla załączników, asynchroniczne generowanie PDF przez Browsershot + kolejka.

4.5

Discovery & Brief AI

Pipeline kolejkowy: transkrypcja Whisper API, ekstrakcja briefa (Claude), analiza stron referencyjnych (Puppeteer + Vision AI), generowanie wireframe (JSON → renderer React). Kwestionariusz: pytania stałe + AI-generowane per branża klienta. Eksport PDF briefa i makiety przez Browsershot. Wymaga działającego Dokumenty (faza 4) — korzysta z PDF exportu. Pełna integracja z SEO możliwa w fazie 6.

5

Powiadomienia

Laravel Reverb setup, Laravel Echo na frontendzie, email notifications, SMSAPI.pl custom channel, preferencje powiadomień per użytkownik, quiet hours. Stopniowe wdrażanie: najpierw email → in-app → SMS.

6

SEO z AI — moduł zaawansowany

TimescaleDB setup, Google Search Console OAuth, Google Analytics 4 API, własny crawler (lub integracja z zewnętrznym API), pipeline kolejkowy (Horizon), integracja AI (Claude API / OpenAI), dashboardy z Recharts, eksport raportów.

Kluczowe decyzje architektoniczne do podjęcia przed startem SEO:

(1) Własny crawler vs zewnętrzne API (ScrapingBee, Screaming Frog API) — wpływa na koszt i złożoność infrastruktury. (2) Dostawca AI — Claude API (Anthropic) dla analizy semantycznej i rekomendacji content; własne modele dla klasyfikacji błędów SEO. (3) Częstotliwość audytów — automatyczne (weekly/monthly) vs on-demand vs oba.

Podsumowanie decyzji technicznych

DECYZJA	WYBÓR	UZASADNIENIE
SPA vs Inertia	**React SPA**	Client panel na subdomenach wymaga API tak czy inaczej
Multi-tenancy	**Single DB + scopes**	Wystarczające na start, prostsze w utrzymaniu
Auth	**Sanctum cookies**	Bezpieczniejszy niż tokeny dla SPA, działa z subdomenami
WebSocket	**Laravel Reverb**	Natywny, bez zewnętrznych zależności (Pusher)
PDF	**Browsershot**	Najwyższa jakość CSS, async przez kolejkę
Edytor	**TipTap v2**	React, TS, rozszerzalny, dojrzały ekosystem
SEO time-series	**TimescaleDB**	Rozszerzenie PostgreSQL — nie zmienia stacku
SMS	**SMSAPI.pl**	Popularny w PL, dobry API, niski koszt
Audit log	**Spatie Activity Log**	Standard de facto w Laravel, zero kodu dla CRUD, polymorphic causer

v1.6.0 — dodano moduł Discovery & Brief AI (sekcja 11), faza 4.5 w roadmapie, aktualizacja flow sprzedaży.

📂 **Dokumentacja ClientOps CRM — powiązane dokumenty**

📋 **Ten dokument**
Specyfikacja Techniczna v1.6
Stack, moduły, decyzje techniczne

🏗️ **Architektura i Model Danych v2.0.0**
clientops-architecture.html
ERD, schematy tabel, przepływy, 27 tabel

🎨 **Design System — Biblioteki v1.0**

clientops-libraries.html · clientops-structure.html

Mapowanie JS, tokeny CSS, komponenty

🛠️ **Przewodnik Implementacji v1.0**

clientops-implementation.html

Kontrolery, migracje, testy, CI/CD

11

AI MODULE

Discovery & Brief AI — onboarding projektowy

Moduł przekształca nagranie rozmowy z klientem w gotowy brief projektowy, makietę wireframe i założenia SEO. Pozycjonowany między Ofertą a Projektem — brief zatwierdza klient w panelu klienckim, dopiero wtedy tworzony jest projekt.

Miejsce w flow sprzedaży:

Lead → Oferta →
Discovery Session
→ Brief (zatwierdzony przez klienta) → Projekt → zasoby projektu. Tabele `discovery_*` należą do warstwy 2.5 — pomostowej między Sprzedażą a Realizacją.

Dwa tryby pracy

TRYB	WYZWALACZ	INTEGRACJA SEO
Nowa strona	Klient nie ma strony lub tworzymy od zera	AI sugeruje słowa kluczowe, strukturę URL, analizę konkurencji
Redesign	Istniejąca strona — modernizacja lub przepisanie	GSC: aktualne rankingi do ochrony, mapa URL, audyt meta/H1

Pipeline kolejkowy

Upload nagrania → TranscribeRecordingJob (Whisper API) → ExtractBriefDataJob (Claude API) →

[redesign] FetchSeoContextJob → GenerateSiteAssumptionsJob → AnalyzeReferenceUrlsJob (Puppeteer + Vision) →

GenerateWireframeJob → NotifyBriefReadyJob

Wszystkie joby na kolejce `ai-processing` przez Laravel Horizon.

Model danych

🎙️ discovery_sessions

`project_id, created_by, mode: new_site|redesign, recording_path, recording_duration_sec, transcript, status: processing|review|approved|exported, existing_url (nullable), approved_at, approved_by`

📋 discovery_briefs

`session_id, client_goals, target_audience, tone_keywords (JSON), required_languages (JSON), integrations_needed (JSON), seo_keywords_suggested (JSON), pages (JSON), sections_per_page (JSON), ai_notes`

🔗 discovery_references

`session_id, url, screenshot_path, analyzed_at, design_notes (text), elements_to_adopt (JSON)`

🖼️ discovery_wireframes

`session_id, version, structure_json (JSON – renderer React), html_preview_path (nullable), generated_at, regenerated_at`

Kwestionariusz Discovery

Formularz wysyłany klientowi przed lub po nagraniu. Trzy warstwy pytań — odpowiedzi wzbogacają prompt AI przy ekstrakcji briefa.

WARSTWA	ŹRÓDŁO	PRZYKŁADY
Ogólne	Stałe, każdy klient	Cel strony, grupa docelowa, ton komunikacji, gotowość treści, języki
Techniczne	Stałe, każdy klient	Integracje (formularz, booking, sklep), hosting, panel CMS, GSC/GA4 dostęp
AI kontekstowe	Generowane per branża	Kancelaria: podstrony prawników, RODO. Stomatolog: rezerwacje, galeria przed/po

🗄️ discovery_question_templates

`layer: general|technical|ai_generated, category, question, is_active, answer_type: text|select|multiselect|boolean|url|scale, options (JSON), order`

📝 discovery_answers

`questionnaire_id, question_template_id (nullable – AI pytania), question_text (kopia w momencie odpowiedzi), layer, answer_text, answer_json`

Makieta (wireframe renderer)

Generowana jako JSON struktury renderowany w React — lekki, interaktywny, bez zewnętrznych narzędzi. Grayscale lo-fi.

```
{
  "pages": [
    {
      "name": "Strona główna",
      "slug": "/",
      "sections": [
        { "type": "hero",          "headline": "...", "cta": true },
        { "type": "features",      "columns": 3 },
        { "type": "testimonials"                    },
        { "type": "cta_banner"                      },
        { "type": "footer"                          }
      ]
    }
  ]
}
```

Klient i admin mogą komentować per sekcja. Export: PDF (Browsershot) lub statyczny HTML.

Integracje techniczne

POTRZEBA	ROZWIĄZANIE	STATUS W STACKU
Transkrypcja audio	OpenAI Whisper API (`whisper-1`)	Nowe
Ekstrakcja briefa + pytania AI	Claude API (`claude-opus-4-6`)	Nowe
Screenshoty referencji + analiza	Puppeteer (Browsershot) + Claude Vision	Browsershot już w stacku
Wireframe renderer	React + własny komponent (JSON → bloki)	Nowe
Kolejki + monitoring	Laravel Horizon, kolejka `ai-processing`	Już w stacku
Pliki audio / screenshoty	Spatie Media Library + S3/MinIO	Już w stacku
PDF briefa i makiety	Browsershot (async, kolejka)	Już w stacku

UI — panel admina

Projekt → zakładka "Discovery"

- Nagraj / wgraj sesję
- Transkrypcja (edytowalna)
- Brief (sekcje edytowalne)
- Kwestionariusz
- Strony referencyjne
- Makieta + komentarze
- Eksport PDF

Panel klienta → "Brief & Makieta"

Wypełnienie kwestionariusza | Podgląd briefa | Komentarze per sekcja | Zatwierdzenie briefa

Faza wdrożenia: 4.5

Po dokumentach (TipTap/PDF), przed powiadomieniami. Pełna integracja SEO ↔ Discovery (tryb redesign) aktywna dopiero w fazie 6. Na starcie Discovery działa samodzielnie bez danych SEO.